Three Column Thinking: A Proof-Carrying, UBP-Compliant Reasoning Protocol with Cross-Domain Isomorphic Compilation — A Comparative Skill Study

Methodology: E. R. A. Craig (DigitalEuan), Auckland, New Zealand
Geometric Language Machine (GLM) — github.com/DigitalEuan/GLM
Skill implementation and experimental study: AI-assisted multi-run comparison
2026-11-01

Abstract

Three Column Thinking (TCT) is a reasoning protocol, originating in the Geometric Language Machine (GLM), in which every computational answer is emitted as a synchronised three-column payload — plain English, exact mathematics, and an executable re-derivation script — and is reported `VERIFIED` only when an independent re-execution agrees exactly. This paper documents an executable implementation of TCT as an agent Skill, extended with a Universal Binary Principle (UBP) discipline (no floats, no randomness, no SHA-256 in computation, standard library only), a platform-dependent Lean 4 verification tier in which unformalised steps are recorded as `sorry` with named development gaps, a post-verification optimisation loop that reduces proof length while preserving independent cross-checks, and a `cent_compiler` (£→¢→Δ) cross-domain isomorphism mode. We evaluate the skill first on two conceptual physics questions ("how does a spring store and release energy?" and "why do things bounce?") across three independent runs: Run A (dola.com's AI system, with the skill's bundled tooling), Run B (Claude.ai, first version of the skill, no bundled scripts, stand-in checker), and Run C (Claude.ai, revised skill with embedded scripts and hardened optimisation loop). All three runs produced exact, independently re-verified answers; the revision closed every defect surfaced by Run B (missing scripts, over-strong Lean tactic claim, silent loss of cross-checks during optimisation, undeclared illustrative givens, toolchain guessing). We then evaluate the updated skill — now carrying a four-conditions "optimal construction" runbook (classification identity, tautology-audited cross-checks, loop acceptance rule, named ledger) — on a broad twelve-question battery spanning eight domains ("what is gravity?", water's anomalous expansion, "what comes after Wednesday?", "what is 1+1?", combinatorics, compound interest, sequence continuation, and parity at a declared $\mathbb{F}_2$ site) in Run D, again on dola.com's AI system, this time with a Lean 4.35 toolchain installed in-environment: the study's first run in which every Lean project

is actually compiled. Run D verified all twelve answers (31/31 Tier-1 spot checks; 12/12 `lake build`), exercised the refusal taxonomy twice with machine-checked justifications, and both corrected and extended the skill's Lean guidance: `decide` and `rfl` fail on `Rat` in pure core (the Nat cross-multiplication pattern is the only kernel-checked route), `omega` and `simp` are present in core where the skill says otherwise, the `ℚ` notation and `Nat.factorial` do not exist in core, and the bundled scaffold's lakefile template is incompatible with Lake 5.0. Finally, four independent case studies produced on a third platform (aistudio.xiaomimimo.com) with the same updated skill — the famous constants, the speed of light, physical isomorphisms, and Maudlin's quantum measurement trilemma — are recorded and audited: their Python tiers reproduce exactly (24/24 scripts, byte-identical bundled tooling, ledger and loop claims all re-verified), while their Lean tiers fail to compile as shipped for the same three stacked reasons Run D documented — "proved, not compiled" is endemic, and the skill's environment guidance is the root cause. Beyond verification, the runs reveal the "why" of TCT: the protocol forces into the open exact-rational witness construction, calculus-free exact witnesses, quantified approximation error, the distinction between counted and independent checks, the load-bearing nature of domain declaration, and refusal-as-proved-claim. Remaining gaps are identified: output-format column fit for multi-line checks, Python-only steps the ledger does not flag as coverage holes, a missing convention for declaring the reading of an ambiguous question, and the scaffold's lakefile template.

1. Introduction

A persistent failure mode of generative reasoning is the confidently wrong answer: a derivation that reads plausibly but is never re-executed. The Geometric Language Machine (GLM) [1] addresses this with Three Column Thinking (TCT, §2.3): every runtime solution is a synchronised three-column payload in which the same logical steps are expressed (1) in plain English, (2) as exact equations over $\mathbb{Z}$, $\mathbb{Q}$, or $\mathbb{F}_2$, and (3) as a dynamically generated Python script that re-derives column 2 in a fresh interpreter with no shared state. A solution is `VERIFIED True` if and only if the independent re-derivation matches perfectly. The GLM authors describe this as "proof-carrying computation, not testing" [1].

This paper reports the design and comparative evaluation of an agent Skill that operationalises TCT. The skill is a single self-contained markdown document (now 586 lines in 14 sections, plus four stdlib-only Python helpers, also embedded in the document) and is intended to load into any LLM agent runtime. We study it first on two deliberately conceptual ("why/how") questions, which force the protocol to separate checkable arithmetic from assumed physics, and we compare three independent runs to measure how the skill's design decisions affect the quality of the result. We then evaluate the updated skill (which adds a four-conditions "optimal construction" runbook, §14 of the skill) on a broad twelve-question battery spanning eight domains, in an environment that — unlike all earlier runs — has a Lean toolchain, so the Lean tier is compiled rather than merely emitted.

2. Background

2.1 The Universal Binary Principle (UBP)

The GLM's UBP (§2.2, §24) enforces exact rational arithmetic as a standing rule with an instrument, not a preference: no floats (`fractions.Fraction` only), no rounding or truncation, standard library only, and re-derived falsifiable facts. Operational bans include no SHA-256 in computation (confined to integrity, directive D3), no XOR except at declared $\mathbb{F}_2$ sites (D9, D11), and no random seeds. No floats is directive D7, checked by parsing every module. The iteration-drift study [1] motivates the rule: under the accumulative recurrence $X_{n+1} = (p+1)/p \cdot X_n - 1/p$, IEEE-754 float64 loses all semantic information by step 200; the drift is deterministic, a hardware consequence, not a model failure.

2.2 Three Column Thinking and reverse TCT

Forward TCT writes the English column from the running step; reverse TCT (GLM Phase 67, §19.3) reverses the direction — the exact mathematics generates the language column through a declared grammar, and a reader parses it back exactly. The skill adopts the reverse-TCT discipline for column 1: each English sentence must be readable back from the corresponding mathematical step.

2.3 The cent_compiler (£→¢→Δ)

The `cent_compiler_v12` subsystem (study GLM-STUDY-CENT-COMPILER-001 [2]) compiles an abstract intent (£) into an intermediate representation of 22 primitives (16 atomic + 6 composite, ¢) and realises them across physical substrates (Δ): electrical, mechanical, fluidic, thermal, optical, digital, spintronic, biochemical. The skill applies only the concept: cross-domain isomorphisms let a problem hard in one domain be solved in an easier one and mapped back. Irrational values are carried as exact rational pulse-density convergents (delta-sigma relaxation), preserving UBP D1.

3. Skill design

The skill is a single self-contained markdown document (586 lines, 14 sections) with four embedded stdlib-only Python helpers. Its pipeline:

1. **Decomposition.** Any chat input — conversational or deterministic — is broken into atomic claims. Each is classified as closed-form (provable), formalisable-but-unproven (`sorry` + named gap), empirical premise (`axiom`), or under-specified (named refusal). For "why/how" questions with no givens, illustrative model values must be explicitly declared and labelled; the mechanism answer must not depend on them.
2. **Three columns.** Plain English (readable back from math), exact mathematics over $\mathbb{Z}/\mathbb{Q}/\mathbb{F}_2$, and a UBP-compliant Python script printing `KEY=VALUE` with `assert` cross-checks between independent derivation paths.

3. **Tier 1 verification.** `verify_column3.py` runs column 3 in an isolated subprocess (exit 0/1/2 = VERIFIED/MISMATCH/ERROR).
4. **Tier 2 verification (Lean).** `scaffold_lean_project.py` emits a lake-build-ready Lean 4 project (pure core by default, `lean-toolchain=leanprover/lean4:stable` so no version guess is needed). Closed arithmetic uses `decide`/`rfl` (zero-import fallback; `norm_num` only if available). `sorry` compiles with a warning, so `lake build` always succeeds; 0 sorry = fully described. `lean_ledger.py` scans and reports the sorry/axiom count. The sorry set is the development-required ledger.
5. **Optimisation loop.** After VERIFIED, propose step reductions (measured by `cent_cost.py` in ¢ primitives). A reduction is accepted only if it remains VERIFIED, produces the identical value, **and preserves or increases the number of independent cross-check paths** — a rule added after Run B showed that "same value" alone permits silent deletion of witness checks.
6. **cent_compiler mode.** Decompose into ¢ primitives, exhibit at least two Δ domain isomorphisms, solve in the easier domain, map back, and ledger licensed (axiom) vs. conjectural (sorry) analogies.
7. **Four-conditions runbook (new in the updated skill, §14).** An optimal run satisfies four exact, checkable conditions, derived by applying TCT to itself: (i) classification identity — every atomic claim lands in exactly one class proved/sorry/axiom/refused, so n = p+s+a+r; (ii) independent cross-checks — if c cross-check paths are written and t tautologies are removed, insight requires (c−t) ≥ 2, where a tautology is any check comparing expressions algebraically equivalent by construction; (iii) optimisation-loop acceptance — a simplification is accepted iff cost decreases **and** the verification-path count is preserved or increased (run one iteration, then stop); (iv) named ledger — every sorry names the exact missing tactic or lemma, every axiom is named, and an unnamed assumed premise is `WOULD_BE_GUESS`. The runbook also records a scanner fix: `lean_ledger.py` now counts only standalone `sorry`/`admit` proof-body lines, so identifiers containing the substring no longer false-positive.

4. Experimental design

Questions (identical across runs). Q1: "How does a spring store and release energy?" Q2: "Why do things bounce?" Both are conceptual, supply no givens, and contain both deterministic sub-claims (energy conservation, coefficient of restitution) and empirical premises (Hooke's law, dissipation).

Runs. - **Run A** — dola.com's AI system (the agent sandbox referred to as "this environment" in the original runs), using the skill's bundled tooling (original and revised versions). Illustrative givens: Q1 k=100 N/m, x=1/5 m, m=1 kg; Q2 e=1/2, v_in=4 m/s, h_0=4 m. - **Run B** — Claude.ai, first version of the skill (no bundled scripts; stand-in checker written by the runner; optimisation loop measured only "same value"). Illustrative givens: Q1 k=200, x_0=1/10, m=1/2; Q2 g=49/5, h=5/2, m=1/2, e=3/4, k_c=61250. - **Run C** — Claude.ai, revised skill (scripts embedded in §13 and materialised; optimisation loop hardened to preserve cross-checks; `decide` fallback; `:stable` toolchain; declared illustrative givens). Same givens as Run B.

Run D (broad-domain battery, updated skill). To evaluate the updated skill — the 14-section document whose §14 runbook formalises the four conditions — on dola.com's AI system with a Lean 4.35.0 / Lake 5.0.0 toolchain installed in-environment, a twelve-question battery was run (T01–T12), keeping the two original physics questions and adding ten questions chosen to stress domains and protocol features the earlier runs never touched:

- T01 "How does a spring store and release energy?" — conceptual physics re-run (k=100, x_0=1/5, m=1, Run A convention);
- T02 "Why do things bounce?" — conceptual physics re-run with engineered rational givens (g=49/5, h_0=5/2, m=1/2, e=3/4, so v_in=7 exactly);
- T03 "What is gravity?" — definition-plus-consequences ("what is" shape);
- T04 "How does water become large when hot and cold?" — anomalous expansion, an almost purely definitional/empirical question (rational witnesses near measured densities);
- T05 "What comes after Wednesday?" — conventional knowledge, cyclic $\mathbb{Z}/7\mathbb{Z}$ arithmetic, an external stdlib calendar anchor;
- T06 "What is 1+1?" — minimal arithmetic, where the domain declaration itself ($\mathbb{Z}$ vs $\mathbb{F}_2$) is load-bearing;
- T07 "How many ways to choose 2 toppings from 5?" — combinatorics, the cross-check showcase;
- T08 "£100 at 5% compound interest for 3 years?" — exact rational finance with a terminating-decimal claim;
- T09 "What comes next: 2, 4, 8, 16, …?" — a refusal-taxonomy stress test (UNDERDETERMINED);
- T10 "What is the area of a circle of radius 1 m?" — an irrational target that must be reported measured, not exact with a declared rational error bound;
- T11 "A light is off; the switch is flipped 17 times — on or off?" — $\mathbb{F}_2$ at a declared XOR site (UBP D9 exercised for the first time in the study);
- T12 "Given V=12 V, I=5 A, R=4 Ω, what is the power?" — the designed INCONSISTENT_GIVENS refusal.

External case studies (the quartet). Beyond the experiment-structured runs, four self-authored TCT studies produced on a third platform, aistudio.xiaomimimo.com, with the same updated skill are recorded and audited as use studies: `famous-constants-tct` (deriving the famous mathematical constants from first principles), `speed-of-light-derivation-tct` (four boundaries of deriving c computationally), `physical-isomorphisms-tct` (five measurable gains of physical→computational isomorphisms, and one refusal), and `maudlin-measurement-trilemma-tct` (Maudlin's quantum measurement trilemma as a truth table). Unlike Runs A–D, these chose their own open-domain questions and shipped their own repositories; the audit (§5.6) re-verified them end-to-end on the Run D environment.

Comparison dimensions. Python verification status; number of independent cross-check paths and executed checks before/after optimisation; Lean proved/sorry/axiom counts; whether `lake build` was run; defects surfaced.

5. Results

5.1 Run A (dola.com’s AI system)

Both questions decomposed into 12 atomic claims: 7 closed-form (proved), 2 symbolic (sorry), 3 empirical (axioms). Tier 1 VERIFIED on both spot-checks (`spring_U=2`, `bounce_h3=1/16`). The Lean scaffold reported 7 proved / 2 sorry / 3 axioms (ready for `lake build`; not compiled — no Lean toolchain in the environment). The optimisation loop reduced the ¢-primitive count 7→4 and passed re-verification. The cent_compiler mode exhibited the spring↔capacitor and mass-spring↔LC-tank isomorphisms, solving the bounce decay geometrically in the electrical domain; the thermal analogy was correctly recorded as conjectural (sorry) rather than asserted.

5.2 Run B (Claude.ai, first skill version)

Both questions answered with exact rationals and Tier 1 VERIFIED (Q1: 6 claims, 2 formalizable + 4 axioms; Q2: 7 claims, 4 formalizable + 3 axioms). The runner wrote a stand-in checker; its first execution failed on a relative-path bug that the runner fixed. Four defects were surfaced: (i) bundled scripts were absent; (ii) the skill claimed core-Lean `norm_num`, which the runner could not confirm (`decide`/`rfl`/`omega` are the core tactics); (iii) the optimisation loop accepted reductions that silently dropped cross-check asserts (Q1 10→4, Q2 12→8) because only “same value” was checked; (iv) illustrative givens were invented without a declared convention. Lean was not built (no `lake`; toolchain version would have had to be guessed).

5.3 Run C (Claude.ai, revised skill)

The runner materialised all four scripts from §13 and used the skill’s own tooling with no stand-in. Tier 1 VERIFIED on every key. Q1: 8 items (3 numeric lemmas, 1 sorry, 4 axioms), 10 named independent cross-check paths, 25 executed checks. Q2: 10 items (6 numeric, 1 sorry, 3 axioms), 12 cross-check paths, 12 checks. Lean scaffolds (Q1: 3 proved / 1 sorry / 4 axioms; Q2: 6 proved / 1 sorry / 3 axioms) were emitted with `:stable` toolchain; `lake build` still not run (no `lake`). The optimisation loop reduced ¢-primitive counts 61→52 (Q1) and 53→48 (Q2) while preserving cross-check paths 10→10 and 12→12 and executed checks 25→25 and 12→12 — the hardened rule prevented Run B’s silent loss.

5.4 Run D (dola.com’s AI system, updated skill, compiled Lean tier)

Setup. The four bundled scripts were materialised verbatim from §13 of the updated skill. Before the battery, the environment’s Lean (4.35.0, Lake 5.0.0) was probed on the exact lemma patterns the skill prescribes; the probes already corrected the skill’s §8 guidance in both directions (details in §7).

Tier 1. All twelve Column-3 scripts ran in isolation; 31/31 pre-declared spot checks (`verify_column3.py --key K --expect V`) returned VERIFIED, including both refusal verdicts (`REFUSED:UNDERDETERMINED`, `REFUSED:INCONSISTENT_GIVENS`), the π-bracket

containment, and the exact bounce ledger (h_3 = 3645/8192 m, total distance 125/14 m, total time exactly 5 s).

Tier 2. All twelve Lake projects were scaffolded from classified specs and **compiled** — the study's first compiled Lean tier: 12/12 `lake build` succeeded, totalling 56 proved / 9 sorry / 25 axioms, with five projects sorry-free (T05, T06, T09, T11, T12). Compilation was not free: the kernel refuted the runner's first cross-multiplied bounce lemma (729·2·4096 = 3645·512·256 is simply false); after correcting it to 729·5·8192 = 3645·4096·2, the build passed. This is the study's first error caught by the Lean tier — Run B's suspicion that "proved" without compilation is weak proof is here made concrete.

Battery results.

Test	Question (domain)	n	p	s	a	r	Lean p/s/a	build
T01	spring (conceptual physics)	12	6	2	4	0	6/2/3	OK
T02	bounce (conceptual physics)	12	7	2	3	0	5/2/3	OK
T03	"what is gravity?" (definition)	12	5	1	6	0	4/1/5	OK
T04	water expansion (empirical)	13	6	1	6	0	4/1/5	OK
T05	after Wednesday ($\mathbb{Z}/7\mathbb{Z}$, calendar)	7	6	0	1	0	4/0/1	OK
T06	1+1 (arithmetic, domain decl.)	5	5	0	0	0	5/0/0	OK
T07	2-of-5 toppings (combinatorics)	6	4	1	1	0	5/1/1	OK
T08	compound interest (finance, $\mathbb{Q}$)	7	5	1	1	0	4/1/1	OK
T09	next term 2,4,8,16 (refusal)	7	5	0	1	1	4/0/1	OK
T10	circle area (irrational, bounds)	11	7	1	3	0	4/1/3	OK
T11	17 switch flips ($\mathbb{F}_2$, XOR site)	7	6	0	1	0	7/0/1	OK
T12	V=12, I=5, R=4 (refusal)	6	4	0	1	1	4/0/1	OK
	Totals	**105**	**66**	**9**	**28**	**2**	**56/9/25**	**12/12**

The classification identity holds exactly: 105 = 66 + 9 + 28 + 2. Overall, 26.7% of all atomic claims are named axioms; the axiom share of the “why/how” and “what is” questions (25–50%: T01 33%, T02 25%, T03 50%, T04 46%) exceeds that of the deterministic questions (0–17%), with one exception — T10 at 27%, inflated because core Lean cannot name π at all and forces its definition, the area formula, and the classical bracket into axioms. T06 (“what is 1+1?”) is the study’s first payload with **zero** axioms and zero sorries: fully described.

Four-conditions runbook. The tautology audit earned its place six times: the 1-based index encoding (T05), the ordered-selection count (T07), the farthings walk (T08), the geometric-decay height variant (T02), the mass-conservation re-assertion (T04), and the Lagrange evaluation (T09) were all reclassified from “independent check” to representation variant. The optimisation loop (one iteration) accepted 7 reductions — every one of which removed an audited tautology or a reporting step, never a witness path — and correctly stopped at the floor for the 5 minimal derivations (T03, T06, T09, T11, T12: dropping any remaining primitive would drop a verification path). Cross-check paths were preserved in every accepted iteration. (The regeneration’s loop re-costing records one accounting refinement: T09 is double-counted in that sentence — it accepted the Lagrange-evaluation removal and then stopped at its floor — and T10, unassigned there, also stops at the floor; 7 accepted + 6 floors − 1 overlap covers all twelve tests exactly.)

5.5 Comparison

Dimension	Run A	Run B (v1)	Run C (revised)	Run D (updated, broad battery)
Platform	dola.com AI system	Claude.ai	Claude.ai	dola.com AI system
Bundled scripts used	yes	no (stand-in)	yes (materialised from §13)	yes (materialised verbatim)
Python VERIFIED	yes	yes	yes (all keys)	yes (31/31 spot checks, 12 questions)
Cross-check paths before→after loop	7→4 primitives; paths preserved	10→4, 12→8 (silent loss)	10→10, 12→12 (preserved)	7 accepted / 5 floor stops; audit removed 6 tautologies; paths preserved in all
Lean tactic	`decide`/`norm_num`	`norm_num` questioned	`decide` fallback; Nat cross-multiply	probes: `decide`/`rfl` fail on `Rat`; Nat cross-multiply; `omega` present in core
Toolchain	`:stable` (no guess)	would have to guess	`:stable` (no guess)	`:stable` → 4.35.0, installed in-environment
Lean proved/sorry/axioms	7/2/3 (combined)	not scaffolded	Q1 3/1/4; Q2 6/1/3	56/9/25 across 12 projects
`lake build` executed	no	no	no	**yes — 12/12 OK** (after a named lakefile workaround)
Illustrative givens	declared	invented, undeclared	declared & labelled	declared & labelled (2 physics questions) / not needed (10)
Defects surfaced	—	4	0 of the prior 4; 4 new gaps (§7)	9 new environment/tooling findings; 1 runner error caught by the Lean kernel

5.6 External case studies: the quartet, recorded and audited

Four independent studies (aistudio.xiaomimimo.com, updated skill) are recorded as case studies; each ships a full TCT repository — eight documentation files, six Column-3 scripts, a Lean project scaffolded from a classified spec, cent decompositions, ledgers, and a verification report — and all four bundle the skill's §13 scripts byte-identically (16/16 files).

Study	Question shape	n = p+s+a+r	Refusal	¢ v1→v2	Lean (as claimed)
famous-constants	constants from first principles	24 = 13+3+7+1	α (`WOULD_BE_GUESS`)	9→6	6/3/7
speed-of-light	four boundaries of deriving c	22 = 11+3+7+1	α from theory (`WOULD_BE_GUESS`)	10→7	5/3/7
physical-isomorphisms	five gains, five preconditions	22 = 13+3+6+0	complexity-class speedups	11→7	9/3/6
maudlin-trilemma	measurement trilemma → 3 doors	20 = 12+3+5+0	—	12→6	10/3/5
	Totals	**88 = 49+12+25+2**	**2**		**30/12/25**

Highlights that extend the paper's findings: π derived three independent ways with overlapping exact intervals below 10^{-25}; the invariant speed derived by three unrelated formalisms (integer wave scheme, GF(2) symplectic Clifford dynamics, Lieb–Robinson v = 2Ja, tight against both) while the dimensionless value is formally refused; measurable isomorphism crossovers (energy 6419×/69×, break-even N = 55/228, reuse T = 21) with complexity-class speedups refused under an explicit extended-Church–Turing axiom; and the measurement trilemma as an exact truth table (7 valuations, 1 excluded, 3 doors) with GRW timings in exact rationals. The quartet also normalises a pattern Run D only glimpsed — the deliberate sorry as a boundary marker: γ's arithmetic type, α from premises, and the Everett emergence gap are open problems written into the formal ledger instead of being hand-waved past.

Audit results (this paper's environment, Lean 4.35.0). The Python tier transfers perfectly: all 24 Column-3 scripts re-verify in isolated subprocesses, and 14/14 expected-value spot checks taken from the studies' own reports (`PURITY=1/2`, `BREAKEVEN_N=55`, `INV_ALPHA_FLOOR=137`, `REFUSAL_REGISTERED=1`, …) match exactly; ledger scans, cent-cost reductions and classification identities all reproduce. The Lean tiers, however, do not compile as shipped — on the toolchain their own `lean-toolchain` pins — for the same three stacked reasons Run D documented independently: the Lake-4-era scaffold lakefile (Lake 5.0 rejects it), the Mathlib-only ℕ/ℚ notations (47 occurrences, which pure core resolves as fresh type variables), and `decide` on rationals (kernel-stuck). Each study reports "`lake build`: not run here; project complete and ready"; the audit shows "ready" was untested and is false on pure core. With the Run-D repairs (lakefile workaround, ℕ→`Nat`/ ℚ→`Rat`, `native_decide` fallback) all four projects build, each carrying exactly its three named sorries as warnings — so their "proved" Lean counts are compiler-checkable but not kernel-checked on pure core. The full audit ships with the case-study record.

6. Why TCT: what the runs revealed

The physics content of both answers is textbook. What is non-obvious is what the protocol forced into the open — the first seven observations come from the physics-

question runs (A–C); the last four (6.8–6.11) are what the broad-domain battery of Run D added. Together they are the “why” of TCT.

6.1 Exact-rational witness construction. Under the no-float rule, illustrative givens are not arbitrary. Both runners engineered them so every quantity stays in $\mathbb{Q}$ and no square root ever appears: $v_in^2 = 49$ (so $v_in = 7$), $\delta^2 = 1/2500$ (so $\delta = 2$ cm), and the contact stiffness $k_c = 61250$ was reverse-engineered to make that true. Where an irrational was unavoidable — v at $x_0/2$ gives $v^2 = 3$ — the runs carried v^2 instead of v. UBP thus drives model construction, not merely computation.

6.2 Calculus-free exact witnesses. The integral $\int kx\,dx$ was cross-checked without invoking calculus. The left and right Riemann sums have closed forms $L = kx_0^2 \cdot (n-1)/(2n)$ and $R = kx_0^2 \cdot (n+1)/(2n)$, so $(L+R)/2 = U$ exactly for every n — the trapezoidal rule is exact for a linear force. The Newton-path check used a central finite difference, exact for the quadratic $v^2(x)$. Verification can be strictly stronger than the textbook derivation it checks.

6.3 Approximations become quantified claims. “Gravity during contact is negligible” is normally handwaved. TCT forced the exact rational bound $mg\delta/KE_in = 1/125$, and the stitched neglect step was itself checked. Every approximation in the chain becomes a named, measurable claim rather than a sentence.

6.4 Counting checks is not independent verification. Run C caught and removed two tautological “cross-checks” (U_half + K_half = U, KE_out + lost = KE_in) that would have inflated the count. The loop’s hardening to count paths, not asserts, was necessary but still insufficient: a cross-check must compare two expressions with disjoint derivation paths. This is the study’s sharpest epistemological finding, and it was discoverable only by running the loop.

6.5 Half of every “why” answer is axiom. Roughly 50% of claims landed as named axioms (Hooke’s law, conservation, dissipation mechanism) in every run. A “why” question is always half arithmetic, half assumed model, and TCT refuses to blur the line. A standard prose answer hides which parts are proved; the ledger makes the boundary explicit.

6.6 The sorry set is tiny and tactic-shaped. Only 1–2 lemmas per question remained sorry, each naming a specific missing tactic (`field_simp`/`ring`, or a concrete `Nat` lemma). All closed arithmetic is `decide`-provable. Full formalisation of elementary physics is therefore blocked by tooling, not by understanding — an encouraging, non-obvious finding for anyone aiming at 0-sorry physics proofs.

6.7 Analogies are self-correcting under the ledger. The cent_compiler mode flagged the thermal-decay analogy as conjectural (exponential in time, not per-cycle) rather than asserting it, while the electrical isomorphism was licensed. Isomorphism validity domains become explicit rather than glossed.

6.8 The domain declaration is load-bearing (T06). “What is 1+1?” has no domain-free answer: 2 in $\mathbb{Z}$, 0 in $\mathbb{F}_2$ (at the declared XOR site), 2 in $\mathbb{Z}/7\mathbb{Z}$. A prose answer silently assumes $\mathbb{Z}$; TCT forces the assumption into the open, and the minimal question still runs the full pipeline — five claims, three genuinely disjoint routes

(definitional successor, disjoint-union cardinality, explicit pairing), zero sorries, zero axioms. The optimisation loop correctly stops immediately at the one-primitive floor.

6.9 A refusal can itself be a proved claim (T09, T12). For "what comes next: 2, 4, 8, 16, …?", the protocol's honest answer is `UNDERDETERMINED` — and the refusal is justified by exact witnesses: the geometric reading (32), the interpolating cubic (30), and Moser's circle-region sequence (31) all match the four givens and disagree, which disproves uniqueness of the continuation. The conventional answer 32 survives only as a conditional on a named simplicity axiom. Likewise T12's INCONSISTENT_GIVENS refusal is executed, not asserted: Column 3 derives all three licensed readings and witnesses their disagreement, and Lean proves each pairwise inconsistency by `decide`. "I cannot answer" becomes a checkable statement about the givens.

6.10 Definitional questions check axiom coherence, not arithmetic (T04, T03). Water's anomalous expansion is almost pure definition-substitution: every "independent" numeric route to the volume ratios is algebraically linked, so the (c−t) ≥ 2 condition can only be met by importing a second axiom base — Archimedes' principle — after which the cross-check verifies the coherence of two axioms (displacement route vs density route agree on the submerged fraction 2750/2997) rather than any arithmetic. "What is gravity?" shows the same shape at larger scale: 50% of its claims are named premises, and the checkable content is the witness triple (t = 5/7 s, v = 7 m/s, h = 5/2 m) satisfying three kinematic relations simultaneously.

6.11 "Measured, not exact" is a working discipline, not a disclaimer (T10). The circle's area cannot be an exact rational, so the no-float rule forces a different mode: two genuinely independent exact-rational interval routes (Archimedes' harmonic/geometric-mean perimeter iteration, and a side-doubling area recursion), each carried entirely in `Fraction` interval arithmetic with bisection square roots, overlap at π ∈ [3.14103…, 3.14271…], sit strictly inside the classical 223/71–22/7 bracket (proved by `decide` on the cross-multiplied integers), and declare the error bound as an exact rational. The Stern–Brocot search for the simplest rational inside the bracket (267/85) is itself exact. The UBP does not forbid irrational answers; it disciplines how they are reported.

6.12 The Python tier transfers; the Lean tier does not — yet (the quartet). The four external case studies are the transfer test Runs A–D could not be: authored independently of the experiment loop, on a third platform, choosing their own questions. Their Python tiers reproduced byte-for-byte and claim-for-claim on the audit machine — the §13 tooling, the KEY=VALUE contract, the ledger discipline and the loop accounting all travel. Their Lean tiers failed to compile as shipped, for exactly the three reasons Run D had documented a day earlier on a different platform. The protocol's split personality at distance: the executable half is robust because it is self-verifying; the formal half is fragile because it makes environment claims the skill asserts rather than probes. The deliberate sorry as boundary marker (γ, α, the Everett gap) is the quartet's own contribution to the method — the honest "we cannot, and here is the named reason" carried into the formal record.

7. Remaining gaps and limitations

What the revision fixed. All four Run-B defects were closed in Run C: the scripts travel embedded in the document (§13) and materialise on first use; the Lean tactic claim was corrected to `decide`/`rfl` with `norm_num` as optional; the optimisation loop now measures verification-path count and rejects any reduction that drops a witness; illustrative givens must be declared and the mechanism answer must be independent of them.

What the updated skill and Run D closed. 1. **Cross-check independence (Run C gap 1) — closed by design and exercised.** The §14 runbook now prescribes the tautology audit — a real check compares two expressions with disjoint derivation paths; anything algebraically equivalent by construction is removed, not counted. Run D's audit removed six such variants across the battery (§5.4), including two the runner had initially written as checks. 2. **`decide` on `ℚ` unconfirmed (Run C gap 2) — resolved empirically, against the skill.** With Lean 4.35.0 compiled in-environment, the probes show the pure-core landscape precisely: the `ℚ` notation itself does not exist in core (it elaborates as a fresh type variable); on the core type `Rat`, both `decide` and `rfl` fail with kernel reduction stuck in `instDecidableEqRat`; `native_decide` works but trusts the compiler. The Nat/Int cross-multiplication pattern that Run C improvised is confirmed as the only kernel-checked route, and Run D used it throughout. The skill's §8/§9 worked-example lemma `(12 : ℚ)^2 / 4 = 36 := by decide` therefore fails in pure core at elaboration, not at the tactic. 3. **The Lean tier was never compiled (Run C gap 5) — closed, with new findings.** Run D compiled all twelve projects (`lake build` 12/12). Two results matter. First, the bundled scaffold's lakefile template is incompatible with Lake 5.0 in two ways — `lean_exe … root := «Main»` fails package configuration, and `lean_lib TCT` no longer globs submodules — requiring a named post-scaffold workaround (`globs := #[.one`TCT.Basic]`); the skill's §13 scaffold needs the same update. Second, compilation has teeth: the kernel refuted the runner's first cross-multiplied bounce lemma, which no amount of Python-side verification would have flagged because the Python value itself was correct — the translation into Lean was the error. "Proved" before Run D meant "a proof was written"; now it means "the kernel checked it".

New findings that correct the skill's §8 guidance (from the Run D probes). `omega` and `simp` are present in pure core Lean 4.35 (the skill lists them as requiring Std/Mathlib), which let Run D prove the general weekday-successor and parity laws symbolically rather than leaving them sorry; `norm_num` and `ring` are confirmed absent; `Nat.factorial` is absent (the scaffold must define its own); `decide` fails on Nat bitwise XOR goals with variables (the regeneration refined the boundary: closed-literal XOR such as `5 ^^^ 3 = 6` does decide, while `∀ a, (a ^^^ 0) = a` does not); and core Lean cannot name `Real.pi` at all, so geometry questions must axiomatise the constant itself (T10's axiom share rose to 27% for exactly this reason). One sharp-edged detail: `omega` proves Int-mod successor laws but fails Nat-mod goals with negated conclusions — goal shape matters when choosing tactics.

Remaining gaps. 1. **Output-format column fit (Run C gap 3, still open).** The prescribed "Python (this step)" table column does not accommodate multi-line check

witnesses; Run D again substituted cross-check names and coverage notes. The skill should permit that substitution explicitly. 2. **Coverage asymmetry (Run C gap 4, still open).** Run D again contains Python-checked steps with no Lean lemma (T01 steps 5 and 7; T03 step 5; T10 steps 4–6 and 8). The ledger reports both tiers' totals but does not flag the difference — a per-step coverage map (which claims have a Lean lemma) is the missing instrument. 3. **No named place for declaring the reading of an ambiguous question.** Three Run D questions required an interpretation declaration before any decomposition: T05 ("after Wednesday" = immediate successor, not any later day), T09 (puzzle convention = conditional answer, not THE answer), T04 ("become large" = expand). The skill's §6 givens block covers givens but not readings; each run improvised a "reading declared" line. The output format should name that block. 4. **Scaffold tooling drift — confirmed at scale by the external case studies.** The bundled `scaffold_lean_project.py` lakefile template is Lake-4-era; until §13 is updated, every user on a current toolchain needs Run D's named workaround. All four external case studies inherited the failure verbatim — plus the ℕ/ℚ notation and decide-on-rationals issues — so their Lean tiers reported "ready" while not compiling on their own pinned toolchain. More generally, Run D and the quartet together show the skill's environment claims (tactic availability, notation, constants) rot with the toolchain and should be phrased as probe-first: state the pattern, verify availability at run time, fall back with a named gap — exactly what Run D did.

On conceptual questions. Roughly half of each answer landed as axioms in every run. This is the method behaving as designed: it cleanly separates what exact arithmetic can check from what physics must assume, and records the latter by name rather than smuggling it in as proved. Run D extends the observation across domains: the axiom share is a measurable signature of question type (25–50% for "why/what is" questions, 0–17% for deterministic ones, π-inflated 27% for T10).

Post-study update (skill v3, same day). After the study, the skill was updated to close all four remaining gaps, and every fix was verified live on the same Lean 4.35.0 / Lake 5.0.0 environment. (i) The output format now permits exactly what Run D improvised: a multi-line Python witness is represented in the table cell by its cross-check name or KEY, with the full witness in the Column-3 script — "never truncate mathematics to fit the column" (gap 1). (ii) The ledger requires a **coverage map**: Lean lemmas cover k of N claims, Python-only steps listed by number with a reason — "a coverage hole is not an error; an unlisted one is" (gap 2). (iii) The output format names a **Reading** block for licensed interpretations of ambiguous wording, declared before decomposition and licensed like axioms, with two conflicting readings routing to `READINGS_DISAGREE` (gap 3). (iv) §8 was rewritten **probe-first**: a new fifth bundled script `lean_probe.py` checks tactic/notation availability at run time against the corrected 4.35.0 baseline (13 probes; on this toolchain 13/13 agree), the wrong `norm_num` / `omega` claims are replaced by the verified landscape table, the failing `ℚ` worked example became the kernel-checked Nat cross-multiplication pattern, and a three-rung degradation ladder governs missing capability — core-available pattern, sorry + named gap, "not run; ready" (gap 4). The bundled scaffold's lakefile now targets current Lake (single `lean_lib` with an explicit glob, no `lean_exe`): a fresh project scaffolded and compiled **as shipped** on Lake 5.0.0 with no workaround — the

failure that all four external case studies inherited is closed at the source. The v3 skill ships with this repository as `skill/SKILL_v3.md`; the version evaluated in Runs A–D is retained verbatim as `skill/SKILL.md`.

8. Conclusion

The TCT skill converts “plausible derivation” into “independently re-verified derivation” across both deterministic and conceptual questions, and its sorry ledger makes the unproved remainder explicit. The comparative runs show that the protocol’s value is not only verification — it is the “why” it forces into the open: exact-rational witness construction, calculus-free exact witnesses, quantified approximation error, and the sharp distinction between a counted check and an independent one. Run D extends this across eight domains and adds three lessons. First, breadth is not dilution: the same 14-section skill produced exact verified answers everywhere from 1+1 to π-bracketing, and its refusal taxonomy answered two questions correctly by refusing, with the refusal itself machine-checked. Second, the Lean tier only tells the truth when it is compiled: Run D’s kernel refuted one of the runner’s own lemmas and falsified the skill’s worked-example tactic pattern while confirming an improvised alternative — evidence that the sorry ledger is provisional until `lake build` runs, and that environment claims in the skill should be probe-first rather than asserted. Third, the four-conditions runbook works: the classification identity closed exactly (105 = 66+9+28+2), the tautology audit removed six would-be checks, and the hardened loop accepted seven simplifications without losing a single witness path. The skill’s design defects are themselves discoverable and closable through exactly the cross-check accounting the protocol prescribes. The principal open work is the output-format column fit, a per-step Lean coverage map, a named declaration block for question readings, and updating the bundled scaffold’s lakefile template for current Lake.

The four external case studies extend the assessment from experiment to transfer: authored independently on a third platform, their Python tiers reproduced exactly under audit while their Lean tiers failed to compile as shipped for the same three reasons — the skill’s executable half is robust because it is self-verifying, and its formal half becomes robust the same way, by probing instead of asserting.

Artifacts

The Run D artifact bundle accompanies this paper. The original payload was lost to an environment recycle; the shipped bundle is a **regeneration** (see its `REGENERATION_NOTES.md`): per-question directories `T01…T12/` each containing `column3.py` (the Column-3 re-derivation, regenerated and re-executed), `spec.json` (lemma inventory locked to the battery table), the scaffolded Lean project `lean_tct/` (built, Lean 4.35.0), and `decomp_before/after.json`; plus the run-level records `RUN_D_SUMMARY.md`, `lean_tier.log`, `optimization_loop.md`, `results.log`, `spot_checks.log` (31/31), and the environment probes `lean_probe/` (15 live probes, `probe_results.md`). Every artifact in it was re-executed in the regeneration environment; the original per-

question payload files (the §6-format three-column answers) are not recoverable, and their analytical content lives in §5.4 and §6.8–6.11 above.

The external case-study record accompanies the paper: `tct_case_studies.md` (the recorded studies and the full audit) and the four quartet repositories as shipped, bundled with the audit logs (`tier1_rerun.log`, `lean_build_audit.log`, `lean_build_repaired.log`, `lean_build_native.log`) and the minimal-repair Lean copies under `repaired/`.

References

[1] E. R. A. Craig (DigitalEuan), The Geometric Language Machine: A Unified Substrate for Exact Computation, Physical Calibration, and Semantic Reasoning (Working Paper, revised 21 Oct 2026), github.com/DigitalEuan/GLM. §2.2 (UBP), §2.3 (Three Column Thinking), §17.4 (three columns), §19.3 (reverse TCT), §24 (standing rules D3/D7/D9/D11).

[2] CENT_COMPILER_ABSORPTION_STUDY (GLM-STUDY-CENT-COMPILER-001, 30 Oct 2026): £→¢→Δ physical compiler, 16 atomic + 6 composite primitives, 31 knowledge-fabric wheels / 72 physical axioms, Lean 4 verified with 0 sorry.

[3] `three-column-thinking` Skill (this study): single self-contained `SKILL.md` (586 lines, 14 sections; the updated version evaluated in Run D adds the §14 four-conditions runbook and the `lean_ledger.py` scanner fix) + `scripts/{verify_column3,scaffold_lean_project,lean_ledger,cent_cost}.py`. Post-study v3 (718 lines, five bundled scripts incl. `lean_probe.py`; probe-first §8, Lake-5 scaffold, Reading block, coverage map) ships as `skill/SKILL_v3.md`; the evaluated version is retained as `skill/SKILL.md`.

[4] Run D artifact bundle (this study): dola.com AI system, 1 Nov 2026 — 12 question directories (T01–T12) with Column-3 scripts, Lean specs and built Lake projects (Lean 4.35.0 / Lake 5.0.0), optimization-loop decompositions, and environment probes and run logs. Regenerated and re-verified live (Tier 1 12/12 + 31/31 spot checks; Tier 2 12/12 built, 56/9/25) after the original payload was lost to an environment recycle; provenance in the bundle's REGENERATION_NOTES.md.

[5] External case-study quartet (aistudio.xiaomimimo.com, 2026), recorded and audited in `tct_case_studies.md`: `famous-constants-tct`, `speed-of-light-derivation-tct`, `physical-isomorphisms-tct`, `maudlin-measurement-trilemma-tct` — four self-authored TCT repositories (88 claims = 49+12+25+2) with byte-identical §13 tooling, re-verified Python tiers, and Lean tiers compiled here for the first time (as shipped: failing; with named repairs: 4/4 building, 3 named sorries each).